

INDEPENDENT EVENT-LOOP AND ASGI REVIEW OF CADELOOP

Date: 2026-08-14T10:28:23Z

Environment: Linux-6.18.35-x86_64-with-glibc2.39; CPython 3.11.15; 3 visible CPUs.

Scope warning: This is an independent bounded review. Repository benchmark files and published numbers were not executed or used as measurements.

EXECUTIVE FINDING

The run cannot honestly name cadeloop, uvloop, rloop, rsloop, or any third-party ASGI server as the measured winner: outbound package/index access was blocked (HTTP CONNECT 403), the checkout had no configured remote, and cadeloop's Rust dependency graph was not cached. Only stdlib asyncio was executable. Any cross-library performance ranking would therefore be fabricated. The useful result is a reproducible stdlib control, a source-level cadeloop assessment, an explicit availability matrix, and a fair validation/report-card framework.

Measured winner in this environment: stdlib asyncio, by default and not by demonstrated superiority. Recommended external follow-up: run this harness in a network-enabled CPython 3.11 Linux host and on Windows 11; do not accept README figures as substitutes.

METHOD AND ROBUSTNESS

- * A new harness under review/ was used; it imports nothing from bench/.
- * Every scenario runs in a fresh subprocess with a hard 15 s timeout, two warmups, five measured samples, and medians.
- * Payloads are deliberately sized for roughly 1-4 seconds per sample group; no individual task is allowed to stale indefinitely.
- * Loopback TCP uses a fixed 1 KiB payload and sequential request/reply to expose loop wakeup overhead rather than load-generator concurrency.
- * The ASGI exchange benchmark measures app/protocol coroutine overhead in-process, not HTTP parsing or a server stack.
- * Results are local controls, not universal capacity claims; no CPU pinning, frequency isolation, or two-host networking was available.

INSTALLATION AND EXECUTION MATRIX

Candidate	Installed/executable	Outcome
---	---	---
stdlib asyncio	yes	fully measured
cadloop	no	build blocked: missing cached proptest; registry access blocked
uvloop	no	PyPI CONNECT tunnel 403
rloop	no	PyPI CONNECT tunnel 403
rsloop	no	PyPI CONNECT tunnel 403
uvicorn	no	PyPI CONNECT tunnel 403
hypercorn	no	PyPI CONNECT tunnel 403
granian	no	PyPI CONNECT tunnel 403
daphne	no	PyPI CONNECT tunnel 403
winloop	not applicable	Windows-only candidate on Linux
tokio/pyo3 bridge	not comparable	runtime integration, not AbstractEventLoop replacement
aiohhttp / Trio / AnyIO	not installed	broader ecosystem comparators, not direct identical surfaces

MEASURED STDLIB CONTROL

Area	Median	Interpretation
call_soon chain	197.0 K callbacks/s	dependency chain / tick latency
call_soon burst	292.5 K callbacks/s	ready-queue drain
zero-delay timer fire	111.2 K timers/s	timer heap + dispatch
sleep(0) chain	194.5 K awaits/s	task rescheduling
task fan-out	46.5 K tasks/s	creation/gather overhead
queue ping-pong	459.6 K exchanges/s	asyncio primitive-heavy path
in-process ASGI exchange	380.2 K requests/s	app call only; no network/parser
TCP echo, 1 KiB	11.63 K msg/s	sequential loopback
TCP p50 / p99	61.8 / 335.7 us	median of run percentiles

THROUGHPUT CHART (HIGHER IS BETTER; UNLIKE UNITS ARE SHOWN ONLY FOR SCALE INSPECTION)

call_soon chain	197.0 K/s	#####
call_soon burst	292.5 K/s	#####
timer fire	111.2 K/s	#####
sleep(0)	194.5 K/s	#####
task fan-out	46.5 K/s	###
queue ping-pong	459.6 K/s	#####
ASGI exchange	380.2 K/s	#####
TCP echo	11.6 K/s	#

VARIABILITY AND CONFIDENCE

The stdlib chain ranged from 157.1 to 215.3 K/s; task fan-out ranged from 34.3 to 48.7 K/s. That spread is large enough that single-run claims would be misleading. Five samples are adequate for a screening run, not publication-grade inference. Cross-library comparisons should report medians, p95/p99 latency, dispersion, failures, and timeout counts?not just best throughput.

CADELOOP EVENT-LOOP SOURCE REVIEW

STRENGTHS

- * The architecture separates Python compatibility from Rust scheduling/reactor state, with hot scheduling methods bound directly to the loop instance.
- * The tick explicitly orders corked write flushes, polling, network callbacks, a second flush, teardown dispatch, and ready callbacks. This is a credible low-latency design.
- * Blocking polls release the GIL while zero-timeout polls retain it; this is sensible, provided all zero-timeout backend paths are provably nonblocking.
- * Cross-thread scheduling captures context and uses a wakeup object rather than relying on polling.
- * The facade covers lifecycle, task/future factories, executors, DNS, signals, async generators, readiness, TCP, UDP, TLS, subprocess support on POSIX, and debug hooks.
- * Configuration validates watermarks, timeouts, limits, worker counts, and experimental RIO opt-in early.

RISKS AND DEFECTS TO INVESTIGATE

1. Support mismatch: package metadata declares CPython 3.11 and Windows production targeting while Linux is a development backend. Claims must be split by OS/backend and wheel availability.
2. Alpha maturity: RIO is explicitly unvalidated and Windows subprocess pipes remain incomplete; these materially reduce drop-in confidence.
3. Private asyncio coupling: the facade uses private asyncio.events, futures, and tasks internals. The 3.11 pin limits exposure but raises upgrade cost.
4. Temporary production traces: server and native tick paths retain environment-gated/worker trace scaffolding tied to a Windows access violation investigation. Treat multi-worker Windows as high risk until resolved.
5. Spin defaults: a 20 us balanced spin trades CPU/energy and noisy-neighbor behavior for latency. Benchmark CPU-seconds/request and idle CPU, not only throughput.
6. Lifespan semantics: startup errors before completion are treated as unsupported lifespan. This mirrors common auto behavior but can mask application startup defects; strict mode would improve operability.
7. HTTP specialization: native HTTP/1.1 can avoid Python parser costs, but protocol breadth must be scored separately: HTTP/2, HTTP/3, proxy headers, trailers, disconnects, WebSocket fragmentation, backpressure, and malformed-input behavior.
8. Eager coroutine stepping: potentially valuable but compatibility-sensitive. Test contextvars, cancellation injection, custom task factories, tracing/profiling, exception groups, and apps that suspend at unusual points.
9. GC freeze default: helpful for stable long-lived heaps, but app-dependent. Measure memory retention and startup object lifetime.
10. Fairness cap: ready batches are capped at 128, which helps I/O fairness; validate timer drift and socket starvation under callback floods.

REQUIRED EVENT-LOOP TEST GRID

Dimension	Cases	Pass criterion
Scheduling	chain, burst, recursive, cancellation, contextvars	semantic parity; no starvation
Timers	same deadline, far future, cancellation storms, clock jump simulation	stable ordering; bounded memory
Threads	1/4/16 producers, close race, context propagation	no loss/deadlock; correct context
TCP	streams/protocols, half-close, reset, pause/resume, backpressure	byte exact; bounded buffers
TLS	handshake failure, cancellation, short write, close_notify, start_tls	no truncation/hang
UDP	connected/unconnected, zero length, truncation, send saturation	correct peer/data/errors
Readiness	pipes/socketpair, replacement callbacks, removal races	asyncio-compatible behavior
DNS	cancellation, cache on/off/expiry, executor shutdown	no leaked futures/threads
Subprocess	stdout/stderr pressure, cancellation, signals, child watcher	no deadlock/zombie
Lifecycle	nested run rejection, close races, asyncgen/executor shutdown	parity and clean resource exit
Debug	slow callbacks, exception handler failure, origin tracking	actionable diagnostics

REQUIRED ASGI/PROTOCOL TEST GRID

Area	Essential cases
HTTP parsing	split boundaries, pipelining, chunk extensions, CL/TE conflicts, malformed fields, smuggling corpus
Request body	streaming, early response, disconnect, max-body enforcement, slowloris
Response	empty body, HEAD, 1xx/204/304, content-length mismatch, chunking, streaming cancellation

WebSocket	accept/reject, ping/pong, fragmentation, UTF-8 errors, close race, max frame/message
Lifespan	unsupported, failure, timeout, cancellation, state copy/isolation
Frameworks	bare ASGI, Starlette, FastAPI, Django ASGI, Channels where supported
Operations	signals, graceful drain, worker death/restart, access logs, stats, TLS reload story
Security	h11 differential corpus, request smuggling, header limits, URI normalization,
fuzz/ASAN/Miri where applicable |

WHICH LIBRARY SHOULD WIN WHERE (PRIOR, NOT THIS RUN'S MEASUREMENT)

Area	Candidate to validate first	Why / caveat
broad compatibility	stdlib asyncio	reference semantics and universal availability
POSIX async throughput	uvloop	mature libuv-based implementation; still measure workload
Windows baseline	stdlib ProactorEventLoop	supported reference; compare winloop/cadeloop IOCP on
real Windows		
raw scheduling experiments	rloop / rsloop	interesting Rust schedulers; incomplete surfaces and
hang gates matter		
HTTP/1.1 minimal ASGI	cadeloop native / granian	native parser/serialization potential; protocol
and framework conformance decide		
mature general ASGI	uvicorn + uvloop (POSIX)	ecosystem and operations maturity
protocol breadth	hypercorn	HTTP/2 and optional HTTP/3 focus; speed may not lead
Django Channels	daphne	ecosystem alignment rather than peak plaintext speed
structured concurrency	Trio/AnyIO	semantics and cancellation model, not drop-in loop speed

REPORT CARD METHODOLOGY

Scores are 0-100 and intentionally separate measured performance confidence from
architecture reputation. Performance confidence is low for every unexecuted candidate in
this environment. Overall is a judgment-weighted screening score (compatibility 20%,
features 18%, maturity 18%, portability 12%, performance confidence 20%, operations 12%),
rounded. It is not a benchmark leaderboard.

| Library / stack | API compat | Features | Maturity | Portability | Perf. confidence | Ops | Overall |
Verdict |
|---|---|---|---|---|---|---|---|
| stdlib asyncio | 100 | 94 | 98 | 100 | 55 | 92 | 90 | safest baseline; only runtime measured here |
| cadeloop native loop | 88 | 84 | 42 | 55 | 25 | 68 | 60 | promising alpha; not buildable in this
offline Linux run |
| cadeloop native ASGI | 80 | 69 | 38 | 55 | 25 | 64 | 55 | high-upside HTTP/1.1 specialization;
validate externally |
uvloop	95	91	94	62	90	88	87	default performance choice on supported POSIX systems
rloop	67	37	30	54	35	34	43	experimental scheduler, not broad drop-in I/O
rsloop	72	52	34	58	35	36	48	experimental; require hang/conformance gates
winloop	87	78	49	24	45	55	59	useful Windows candidate, platform-specific
uvicorn + asyncio	96	91	95	96	58	92	88	mature general ASGI baseline
uvicorn + uvloop	94	91	94	61	88	91	88	strong POSIX production default
hypercorn	92	96	87	94	57	83	85	best protocol breadth (HTTP/2/3 dependent on extras)
granian	86	87	76	77	82	79	81	compelling Rust server; independently load-test
daphne	91	87	90	92	45	83	81	mature Channels/WebSocket choice, not speed-first
tokio/pyo3 async bridge	38	35	77	78	50	45	54	integration primitive, not an asyncio
loop replacement								
aiohttp server (non-ASGI)	45	86	94	94	70	88	80	strong native ecosystem; not direct

ASGI substitute |
| Trio/AnyIO (structured concurrency) | 42 | 78 | 89 | 90 | 52 | 84 | 75 | semantics choice, not drop-in
event-loop contender |

FINAL CONCLUSION

Cadeloop shows technically serious design work: a native scheduler/reactor, bounded ready batches, explicit GIL boundaries, transport backpressure, native HTTP/ASGI, and substantial compatibility scaffolding. Its largest obstacles are validation, not ideas: the package could not be built here, the production target differs from this host, RIO remains experimental, and a Windows worker crash investigation is visible in source. The unbiased decision today is do not declare cadeloop fastest from this run. Use stdlib for maximum compatibility, uvloop/uvicorn as the first mature POSIX performance baseline, Hypercorn where protocol breadth dominates, and independently validate cadeloop native and Granian for HTTP/1.1 throughput.

REPRODUCTION AND ARTIFACTS

- * review/run_independent_review.py - bounded harness.
- * review/independent-results.json - all raw samples and environment metadata.
- * review/cadeloop-independent-review.pdf - this report.
- * Network/install attempt: uv pip install ... failed with CONNECT tunnel 403.
- * Build attempts: cargo build --release -p cadeloop-pyshim could not reach crates.io; offline resolution lacked proptest.